

レポート課題 10

05-191009 奥村真司

問 1

動作例

```
?- bloodrelative(iwao,X).
X = sanae ;
X = kobo ;
false.

?- bloodrelative(kobo,X).
X = kobo ;
X = miho ;
X = kobo ;
X = sanae ;
X = kobo ;
X = sanae ;
X = kobo ;
X = koji ;
X = sanae ;
X = iwao ;
X = mine ;
false.
```

考察

はじめは定義通り `bloodrelative(X,Y) :- ancestor(X,Z),ancestor(Y,Z).` とした。実行結果は kobo 自身や他の人も何度も出力されるが探索上仕方ないとした。この定義では kobo と iwao などが血縁関係にならないので X,Y のどちらか一方がもう一方の祖先の場合も血縁関係となるように修正した。動作例は修正後のものである。

問 2

動作例

```
?- mult(s(s(z)),s(s(s(z))),X).
X = s(s(s(s(s(s(z)))))).
?- mult(X,s(s(s(z))),s(s(s(z)))).
X = s(z) ;
(don't stop)
-----
```

```
?- mult(s(z),X,s(s(s(z)))) .
X = s(s(s(z))) ;
false.
```

考察

定義は,

```
mult(z,_,z).
mult(s(X),Y,Z) :- mult(X,Y,W),add(Y,W,Z).
```

である。add は講義スライドのものを用了。第一引数を小さくしていきながら 1 行目の base ケースに帰着される。なので、第二引数、第三引数に対する問い合わせは base ケースに到達し、探索が停止する。逆に第一引数に対する問い合わせは停止しなかった。また停止はしなかったがスタックオーバーフローは起こさなかったので探索で無限ループしていると考えられる。

問 3

動作例

```
?- reverse([1,2,3],X).
X = [3, 2, 1].

?- reverse(X,[1,2,3]).
X = [3, 2, 1] ;
(don't stop)

-----
(add length check ver.)
?- reverse([1,2,3],X).
X = [3, 2, 1].

?- reverse(X,[1,2,3]).
X = [3, 2, 1] ;
false.
```

考察

コンスでは先頭が簡単に取り出せる。reverse の判定をしたいのでリストの最後尾を取り出したくなった。なので

$$\text{pushback}(X,Y,Z) \Leftrightarrow \text{リスト } X \text{ の最後尾に } Y \text{ を挿入したら } Z$$

という述語を定義した。reverse の定義は

```
reverse([], []).
reverse([A|X],Y) :- reverse(X,Z),pushback(Z,A,Y).
```

とした。第一引数のリストは長さが短くなって base ケースに到達するので第一引数への問い合わせは止まる。しかし第二引数への問い合わせは止まらなかった。ここで、一方のリストが与えられれば、もう一方のリストの長さは決定するので探索空間を狭められる (長さの制約のみでは依然と

して解の候補は無限だが) と思い samelength という述語を用いて第二引数との長さのチェックを入れてみたところ、第二引数への問い合わせでも停止するようになった。このことから、Prologでの探索では未決定なものについて仮の値を定めてカンマのつぎの述語のチェックをしてくれていると考えられる。実際、reverse(X,Y) のように両引数について問い合わせをすると、以下のように仮の値と思われる値がインタプリタから出力されることを確認できた。

```
?- reverse(X,Y).
X = Y, Y = [] ;
X = Y, Y = [_3280] ;
X = [_3280, _3286],
Y = [_3286, _3280] ;
X = [_3280, _3298, _3286],
Y = [_3286, _3298, _3280] ;
X = [_3280, _3298, _3304, _3286],
Y = [_3286, _3304, _3298, _3280] ;
X = [_3280, _3298, _3316, _3304, _3286],
Y = [_3286, _3304, _3316, _3298, _3280] ;
.....
```

問 4

動作例

```
?- hamilton([1,2,3,4],[[1,2],[2,3],[3,4],[4,1]]).
true ;
true ;
true ;
true ;
false.
?- hamilton([1,2,3,4],[[1,2],[2,3],[3,4]]).
true ;
false.
?- hamilton([1,2,3,4],[[1,2],[2,3]]).
false.
```

考察

V を頂点番号のリストとする。各有向辺 $(u,v) \in E$ に対して $[u,v]$ のリストを E とする。(動作例参照) はじめ、

$$\text{permutation}(X,P) \Leftrightarrow "P \text{ は } X \text{ の順列}"$$

と

$$\text{ispath}(P,E) \Leftrightarrow "P \text{ はパス}"$$

という predicate を作ろうと思ったがうまく実装できなかったので諦めた。

$$\text{choose}(X,Y,Z) \Leftrightarrow Z \text{ のどこかに } Y \text{ を挿入すると } X$$

$$\text{in}(X,Y) \Leftrightarrow X \text{ はリスト } Y \text{ に含まれる}$$

という predicate を用意して、探索した。hamiltonsub は残りの頂点集合、現在いる頂点、辺集合 E をもらって残りを探索する。hamilton はスタートする頂点を全て試しながら hamiltonsub を呼ぶ。全探索するので、存在するハミルトンパスの数だけ true が出る。

発展 1

動作例

```
?- interpreter("++++++>++++++>++++++>++++>+<<<-]
>.>+.+++++.+.+.>++++.<++++++>++++.>+.+.-----.--
-----.>+.>+.").
Hello World!
true ;
false.

/* solution of https://atcoder.jp/contests/abc131/tasks/abc131_a */
?- interpreter(">++++>, <+++[->, [-<->+<] <[[-] <<->]>>
[-<<+>]<<<]+<[[-] >++++++[-<++++>]<.[--<++++>]<--
.+++>]>[++++++[-<++++++>]<-.+++[->++++<]>..[-<<+>]
]<<----->]<<.").
|: 3667
Bad
true .

?- interpreter(">++++>, <+++[->, [-<->+<] <[[-] <<->]>>
[-<<+>]<<<]+<[[-] >++++++[-<++++>]<.[--<++++>]<--
.+++>]>[++++++[-<++++++>]<-.+++[->++++<]>..[-<<+>]
]<<----->]<<.").
|: 8080
Good
true .

?- interpreter("abc").
false.

?- interpreter("<+").
false.
```

考察

以下はきちんと意味論が展開されていないためふんわりとした説明である。チューリング完全の定義を調べて見た。任意のチューリングマシンでの任意の入力に対する計算が可能な計算モデルを万能チューリングマシンといい、万能チューリングマシンと同じ計算能力を持つときその計算モデルをチューリング完全というそうである。Brainfuck は命令数の非常に少ない言語であり、チューリング完全であることが知られている言語である。Brainfuck のインタプリタを Prolog で書くことで、Prolog がチューリング完全であることを示すという方法をとった。以下でインタプリタの実装についてと、Prolog の理解が甘かったために実装中に困った点について説明する。

インタプリタの実装

メモリの状態を表す整数のリストと、ポインタとプログラムカウンタでマシンの状態を表せる。手続き型の言語のインタプリタなので、Prolog の述語は全て引数のうち前のいくつかを決めてやることで後半の全てが決定するような関数のようなものばかりで構成した。Parser の実装は面倒

なので標準である述語 `string_to_list` を用いた。実装したいいくつかの述語について説明する。

- `interpreter(X)`
`brainfuck` のソースとして正しいものかどうかを返す熟語。ソースコードを文字列として受け取って実行する。エラーの場合や `Syntax error` の場合 `false` となる。文字列をパースして整数のリストにし、下の `exec` を初期状態で呼び出す。
- `exec(Code,Length,Counter,Memory,Pointer)`
プログラムカウンタが `Counter` で、メモリとポインタが `Memory.Pointer` の状態から最後まで正しく実行できるかどうかの述語。各命令に合わせてマシンの状態を変化させ、再帰的に `exec` を呼び出す。`Counter` が `Code` の最後まで来たら、`true` となり、プログラムに問題があれば `false` となる。
- `increase/decrease memory,increase/decrease pointer,correspond_start/end_position`
各命令を実行した後のマシンの状態を求めるために補助的に用意した述語 (関数)。名前から行っていることはわかると思うので詳細は実装を見ていただきたいです。

Prolog の理解

実装をする際に Prolog の整数演算の扱いの理解が甘く悩んだのでそこで得た知見について説明する。Prolog=は値の比較ではなく単一化を表すということを理解していなかったため、`1+2=3` が `false`. で困惑した。整数演算を評価した結果を比較したい場合は `<`, `>`, `=:=` などの演算子を用いる必要があることを学んだ。また整数演算の評価結果を未決定の変数が等しいことは `X is 1+2` のようにすることでできるとわかった。

Brainfuck の言語仕様

検証のために実際の Brainfuck 処理系との挙動を比較した際に気づいたことについて、非本質的ではあるが説明する。はじめメモリ (テープ) 双方向に無限の長さをもつと思っていたので初期位置よりも添字の小さいメモリにもアクセスできるようにしていたが、実際は片方向にのみだけ無限で、動作例の 4 つめのようなものは実行時エラーを起こすとわかったのでその仕様に合わせた。

Brainfuck のメモリのデータは符号なし 8bit 整数型であるので、その仕様に合わせた。動作例の 2,3 つ目のコードは 0 をデクリメントすると 255 になることを用いているのでそこで気づいた。